Name : AKASH SAMPATH

Title : Mock Project

Creation of Project Structure : Using Visual Studio :

Flow Diagram :

```
docker-multi-app
 ├── backend
 |   ├── Dockerfile
 |   ├── server.js
 |   └── package.json
 ├── frontend
 |   ├── Dockerfile
 |   └── index.html
 └── docker-compose.yml
```

Procedure : Commands :

Open terminal in visual studio code : Give the following commands :

- mkdir docker-multi-app
- cd docker-multi-app
- mkdir backend
- mkdir frontend

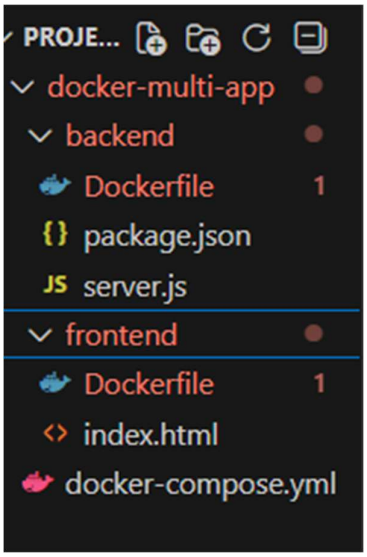
Create a file : Inside docker-multi-app :

- By clicking create new file name as docker-compose.yml

Inside the Folder : Create another sub files :

- **In backend :** Dockerfile : server.js : package.json
- **In frontend :** Dockerfile : index.html

Output : Folder and File Structure : Creation : docker-multi-app :



Step 1 : Creation of Backend App : Node.js API :

- **In backend :** package.json : server.js : Dockerfile
- Paste the code in the package.json : server.js : Dockerfile

Code : backend : package.json

```
{
  "name": "backend",
  "version": "1.0.0",
  "dependencies": {
    "express": "^4.18.2",
    "pg": "^8.11.0"
  }
}
```

Code : backend : server.js

```
const express = require("express");
const { Pool } = require("pg");

const app = express();

const pool = new Pool({
  host: "db",
  user: "postgres",
  password: "postgres",   // Update password used in postgres
  database: "testdb",     // Update DB used in postgres
  port: 5432
});

app.get("/", async (req, res) => {
  const result = await pool.query("SELECT NOW()");
  res.json({
    message: "Backend working ✅",
    time: result.rows[0]
  });
});

app.listen(5000, () => console.log("Backend running on port 5000"));
```

Code : Backend : Dockerfile :

```
FROM node:18

WORKDIR /app

COPY package.json .
RUN npm install

COPY server.js .

EXPOSE 5000

CMD ["node", "server.js"]
```

Step 2: Creation of Frontend App :

- **frontend :** index.html : Paste code :
- **frontend :** Dockerfile : Paste code :

Code : frontend : index.html :

```
<!DOCTYPE html>
<html>
<body>
<h2>Frontend Page</h2>

<button onclick="callAPI()">Call Backend</button>

<pre id="output"></pre>

<script>
function callAPI() {
  fetch("http://localhost:5000")
    .then(r => r.json())
    .then(d => document.getElementById("output").innerText =
      JSON.stringify(d, null, 2));
}
</script>
```

```
</body>
</html>
```

Code : frontend : Dockerfile :

```
FROM nginx:alpine

COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80
```

Step 3 : Creation of Docker Compose File :

- Create file name as docker-compose.yml under folder project docker-multi-app
- Paste the code : In docker-compose.yml
- Update password in postgres code

Code : docker-compose.yml :

```
version: "3.9"

services:

  db:
    image: postgres:15
    container_name: postgres_db
    environment:
      POSTGRES_PASSWORD: admin
      POSTGRES_DB: testdb
    ports:
      - "5432:5432"

  backend:
    build: ./backend
    container_name: backend_app
    ports:
      - "5000:5000"
    depends_on:
      - db

  frontend:
    build: ./frontend
    container_name: frontend_app
    ports:
```

```
    - "8080:80"
  depends_on:
    - backend
```

Note : Save all commands everything :

Step 4 : Build and Start Everything using docker compose :

- Open and keeping running docker desktop

Give the following command :

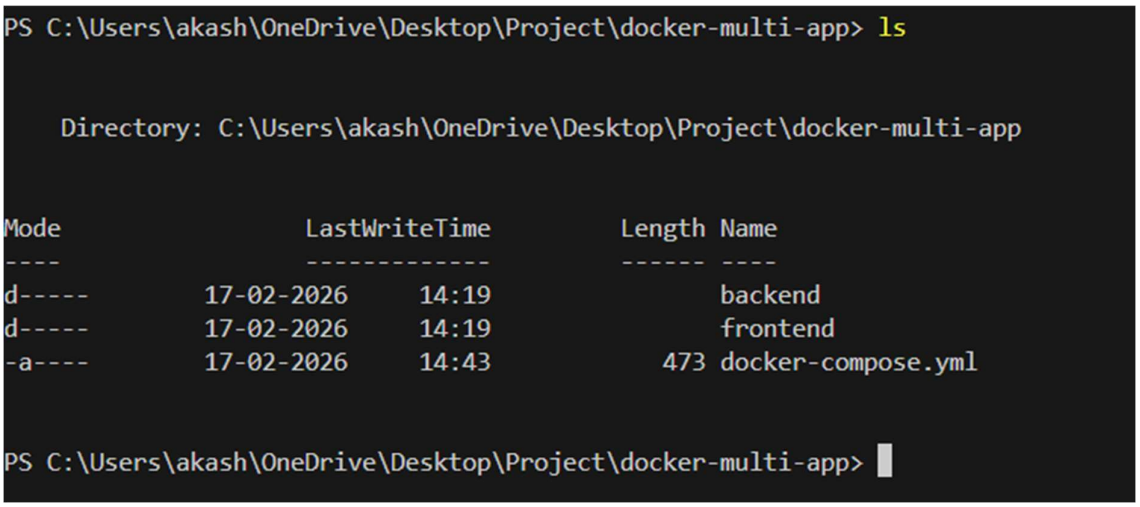
Opening the project folder :

- cd docker-multi-app

Verifying if correct folder

- ls

Output : Verifying :

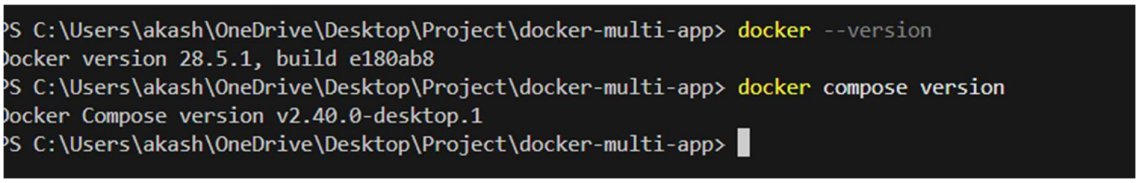


Verify Docker installed :

Commands :

- docker –version
- docker compose version

Output : Verify Docker installed :

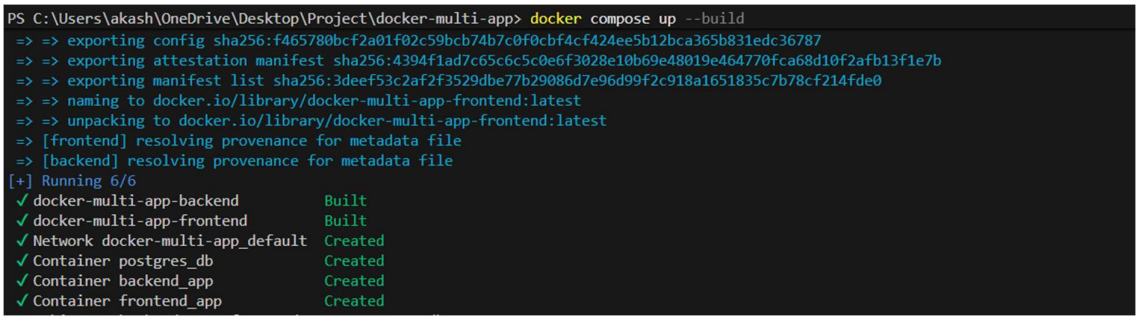


Build images to start all containers :

Commands :

- docker compose up –-build

Output : Using : docker compose up –-build : Creation :

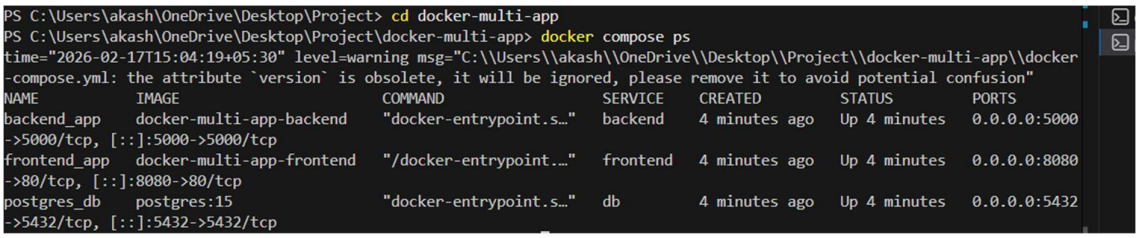


Confirming containers running : New terminal :

Commands :

- cd docker-multi-app
- docker compose ps

Output : Containers : Running :



Step 5 : Test Applications :

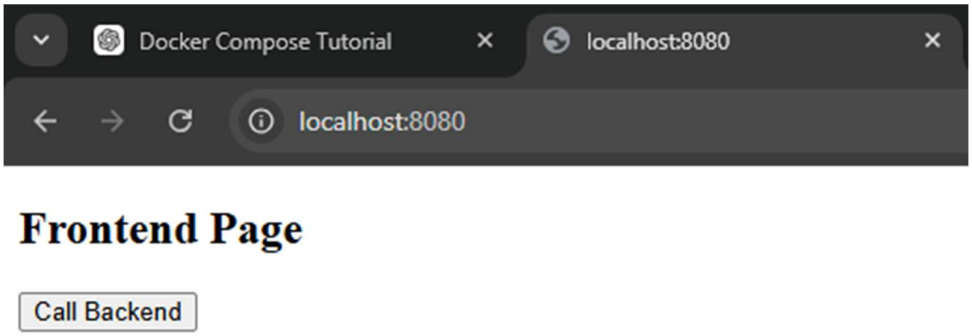
Frontend :

- **Open browser :** http://localhost:8080

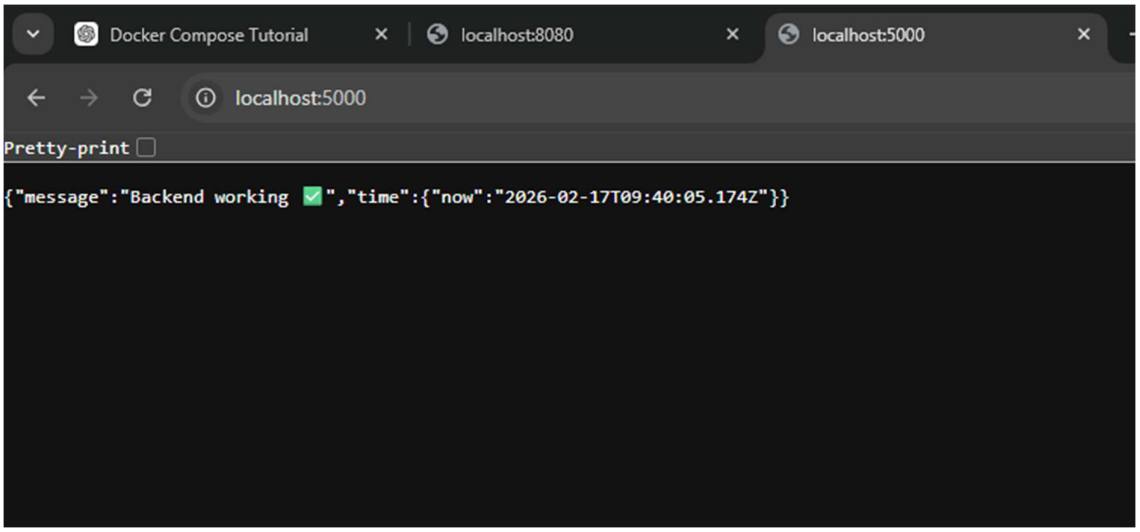
Backend :

- **Open browser** : http://localhost:5000

Output : Test : Frontend :



Output : Test : Backend :



Step 6 : Checking Running Status : Commands :

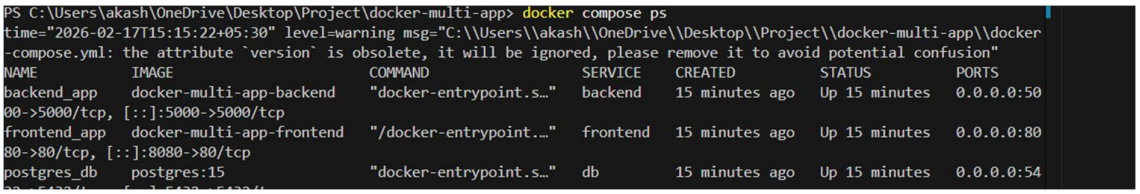
Show running containers :

- docker compose ps

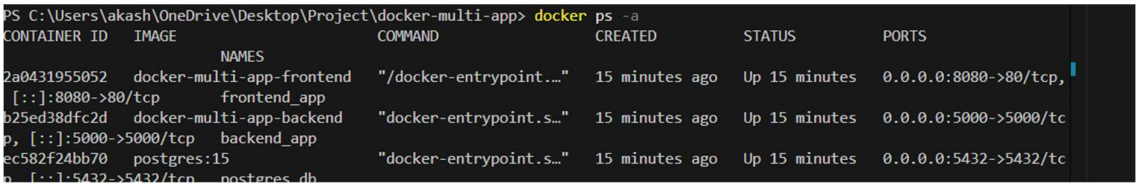
Show all containers :

- docker ps -a

Output : Showing running containers :



Output : Showing all containers :



Step 7 : Checking Logs : Commands :

All services logs :

- docker compose logs

Backend logs :

- docker compose logs backend

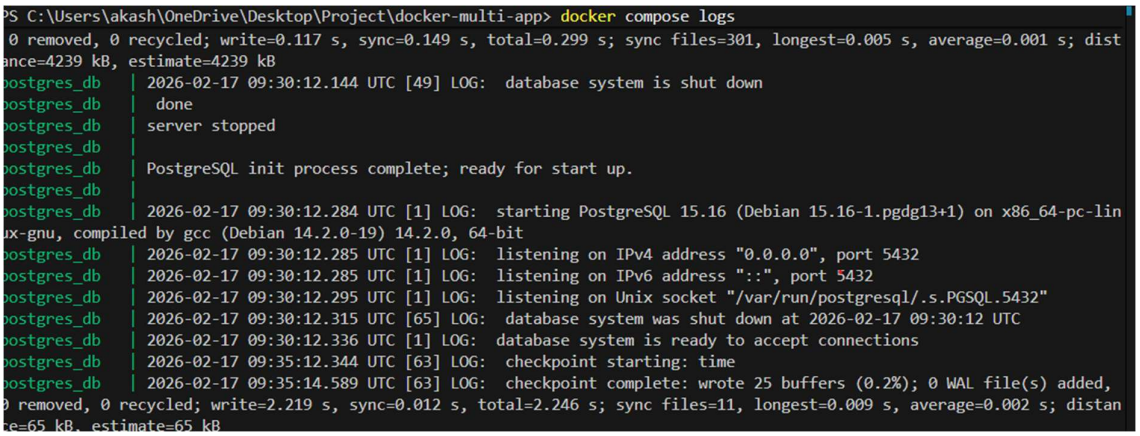
Frontend logs :

- docker compose logs frontend

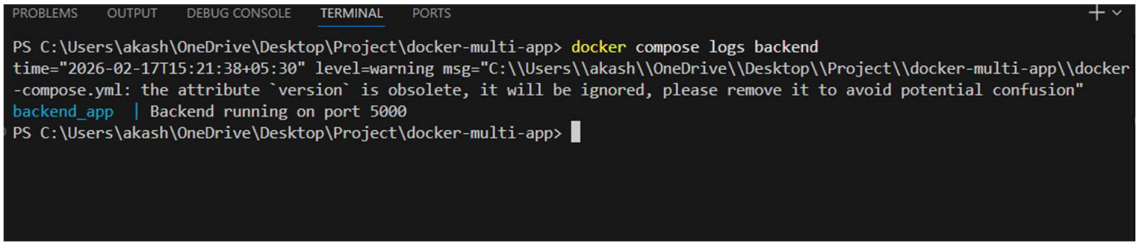
Database Logs :

- docker compose logs db

Output : All services logs :



Output : Backend logs :



Output : Frontend logs :

```
PS C:\Users\akash\OneDrive\Desktop\Project\docker-multi-app> docker compose logs frontend
time="2026-02-17T15:22:49+05:30" level=warning msg="C:\\Users\\akash\\OneDrive\\Desktop\\Project\\docker-multi-app\\docker
-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
frontend_app  | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
frontend_app  | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
frontend_app  | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
frontend_app  | 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
frontend_app  | 10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
frontend_app  | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
frontend_app  | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
frontend_app  | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
frontend_app  | /docker-entrypoint.sh: Configuration complete; ready for start up
frontend_app  | 2026/02/17 09:30:09 [notice] 1#1: using the "epoll" event method
frontend_app  | 2026/02/17 09:30:09 [notice] 1#1: nginx/1.29.5
frontend_app  | 2026/02/17 09:30:09 [notice] 1#1: built by gcc 15.2.0 (Alpine 15.2.0)
frontend_app  | 2026/02/17 09:30:09 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
frontend_app  | 2026/02/17 09:30:09 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
frontend_app  | 2026/02/17 09:30:09 [notice] 1#1: start worker processes
frontend_app  | 2026/02/17 09:30:09 [notice] 1#1: start worker process 30
frontend_app  | 2026/02/17 09:30:09 [notice] 1#1: start worker process 31
```

Output : Database Logs :

```
PS C:\Users\akash\OneDrive\Desktop\Project\docker-multi-app> docker compose logs db
time="2026-02-17T15:23:53+05:30" level=warning msg="C:\\Users\\akash\\OneDrive\\Desktop\\Project\\docker-multi-app\\dock
-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
postgres_db  | The files belonging to this database system will be owned by user "postgres".
postgres_db  | This user must also own the server process.
postgres_db  |
postgres_db  | The database cluster will be initialized with locale "en_US.utf8".
postgres_db  | The default database encoding has accordingly been set to "UTF8".
postgres_db  | The default text search configuration will be set to "english".
postgres_db  |
postgres_db  | Data page checksums are disabled.
postgres_db  |
postgres_db  | fixing permissions on existing directory /var/lib/postgresql/data ... ok
postgres_db  | creating subdirectories ... ok
postgres_db  | selecting dynamic shared memory implementation ... posix
postgres_db  | selecting default max_connections ... 100
postgres_db  | selecting default shared_buffers ... 128MB
postgres_db  | selecting default time zone ... Etc/UTC
postgres_db  | creating configuration files ... ok
postgres_db  | running bootstrap script ... ok
```

Step 8 : Entering Running Container :

Backend Container Shell :

Commands :

- docker exec -it backend_app sh

Testing Connection from backend container :

Commands:

- curl http://localhost:5000

Output : Test Connection : Backend Container :



Step 9 : Stopping Everything and Removing and Restarting :

Commands :

- docker compose down
- docker compose down -v
- docker compose up